

Programmazione a oggetti

Classi e oggetti

I concetti essenziali della programmazione a oggetti in PHP.

Perche usare gli oggetti

Quando un programma cresce, variabili e funzioni sparse possono diventare difficili da seguire. La programmazione a oggetti aiuta a tenere insieme dati e comportamenti collegati.

Classe e oggetto

Una **classe** è un modello. Un **oggetto** è una copia concreta creata da quel modello.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;
}

$quaderno = new Prodotto();
$quaderno->nome = "Quaderno";
$quaderno->prezzo = 3.50;

echo $quaderno->nome;
```

`new Prodotto()` crea un oggetto.

Proprieta

Le proprieta sono variabili dentro una classe.

```
public string $nome;  
public float $prezzo;
```

`public` significa che puoi leggere e modificare la proprietà dall'esterno.

Metodi

I metodi sono funzioni dentro una classe.

```
<?php  
class Prodotto {  
    public string $nome;  
    public float $prezzo;  
  
    public function descrizione(): string {  
        return $this->nome . " costa " . $this->prezzo . " euro";  
    }  
}  
  
$prodotto = new Prodotto();  
$prodotto->nome = "Penna";  
$prodotto->prezzo = 1.20;  
  
echo $prodotto->descrizione();
```

`$this` indica l'oggetto corrente.

Quando iniziare a usarli

All'inizio puoi scrivere programmi semplici con variabili, array e funzioni. Gli oggetti diventano utili quando vuoi rappresentare concetti del progetto: utenti, prodotti, ordini, articoli.

Oggetti e array associativi

Un array associativo può descrivere un prodotto:

```
<?php
$prodotto = [
    "nome" => "Penna",
    "prezzo" => 1.20,
];
```

Una classe rende esplicito che quei dati appartengono a un concetto preciso.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;
}
```

La classe dice: un prodotto ha un nome e un prezzo. Questo aiuta quando il progetto cresce.

Creare piu oggetti

```
<?php
$penna = new Prodotto();
$penna->nome = "Penna";
$penna->prezzo = 1.20;

$quaderno = new Prodotto();
$quaderno->nome = "Quaderno";
$quaderno->prezzo = 3.50;
```

`$penna` e `$quaderno` sono due oggetti diversi creati dallo stesso modello.

Errore comune: dimenticare new

Per creare un oggetto devi usare `new` .

```
<?php
$prodotto = new Prodotto();
```

Senza `new`, non stai creando un'istanza della classe.

Costruttori e visibilit 

Come inizializzare oggetti e controllare l'accesso a propriet  e metodi.

Il costruttore

Il costruttore   un metodo speciale chiamato automaticamente quando crei un oggetto.

```
<?php
class Prodotto {
    public string $nome;
    public float $prezzo;

    public function __construct(string $nome, float $prezzo) {
        $this->nome = $nome;
        $this->prezzo = $prezzo;
    }
}

$prodotto = new Prodotto("Quaderno", 3.50);
```

`__construct` prepara l'oggetto con i dati iniziali.

Visibilit 

La visibilit  decide da dove puoi usare propriet  e metodi.

- `public` : accessibile anche dall'esterno
- `private` : accessibile solo dentro la classe
- `protected` : accessibile dentro la classe e nelle classi figlie

Private

```
<?php
class Conto {
    private float $saldo = 0;

    public function deposita(float $importo): void {
        if ($importo > 0) {
            $this->saldo += $importo;
        }
    }

    public function leggiSaldo(): float {
        return $this->saldo;
    }
}
```

`$saldo` è privato: dall'esterno non puoi modificarlo direttamente. Devi passare dai metodi.

Incapsulamento

L'incapsulamento significa proteggere i dati interni di un oggetto e offrire metodi chiari per usarli.

Questo aiuta a evitare valori impossibili, come un deposito negativo.

Getter e setter

Un **getter** legge un valore. Un **setter** lo modifica controllando prima se va bene.

```
public function leggiSaldo(): float {
    return $this->saldo;
}
```

Non servono sempre, ma sono utili quando vuoi controllare l'accesso ai dati.

Promozione delle proprietà nel costruttore

In PHP moderno puoi scrivere costruttori più compatti.

```
<?php
class Prodotto {
    public function __construct(
        public string $nome,
        private float $prezzo
    ) {
    }
}
```

Questa sintassi crea le proprietà e le valorizza nello stesso punto. È comoda, ma all'inizio va letta con calma:

`public string $nome` dentro il costruttore diventa una proprietà pubblica della classe.

Setter con controllo

Un setter serve quando vuoi permettere una modifica, ma solo se il valore è valido.

```
<?php
class Prodotto {
    private float $prezzo;

    public function cambiaPrezzo(float $prezzo): void {
        if ($prezzo <= 0) {
            return;
        }

        $this->prezzo = $prezzo;
    }
}
```

Qui impedisce prezzi uguali o minori di zero.

Regola pratica

Usa `private` per i dati che non devono essere modificati liberamente. Offri metodi pubblici quando vuoi controllare come quei dati vengono letti o cambiati.

Ereditarieta, interfacce e trait

Come riusare e organizzare comportamento tra classi PHP.

Prima la soluzione semplice

Ereditarieta, interfacce e trait sono strumenti utili, ma non servono in ogni programma. Prima prova con classi piccole e metodi chiari. Usa questi strumenti quando risolvono un problema reale.

Ereditarieta con extends

L'ereditarieta permette a una classe di partire da un'altra classe.

```
<?php
class Utente {
    public function saluta(): string {
        return "Ciao";
    }
}

class Admin extends Utente {
    public function pannello(): string {
        return "Area amministrazione";
    }
}

$admin = new Admin();
echo $admin->saluta();
```

`Admin` eredita il metodo `saluta` da `Utente` .

Interfacce con implements

Un'interfaccia descrive quali metodi una classe deve avere.

```
<?php
interface Notificabile {
    public function invia(string $messaggio): void;
}

class Email implements Notificabile {
    public function invia(string $messaggio): void {
        echo "Email: $messaggio";
    }
}
```

L'interfaccia non spiega come fare il lavoro. Stabilisce il contratto.

Trait

Un trait permette di riusare metodi in più classi.

```
<?php
trait LoggaAzioni {
    public function log(string $messaggio): void {
        echo "Log: $messaggio";
    }
}

class Ordine {
    use LoggaAzioni;
}
```

Quando usarli

Usa `extends` quando una classe è davvero una versione più specifica di un'altra. Usa un'interfaccia quando classi diverse devono rispettare lo stesso contratto. Usa un trait per condividere piccoli comportamenti, senza costruire una gerarchia complicata.

Un esempio di interfaccia con piu classi

Il valore di un'interfaccia si vede quando classi diverse possono essere usate nello stesso modo.

```
<?php
interface Pagabile {
    public function paga(float $importo): void;
}

class CartaCredito implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con carta: $importo";
    }
}

class Bonifico implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con bonifico: $importo";
    }
}
```

Entrambe le classi promettono di avere un metodo `paga` .

Quando evitare l'ereditarieta

Non usare `extends` solo per riusare due righe di codice. L'ereditarieta crea un legame forte tra classi. Se il legame non rappresenta davvero un rapporto "e un tipo di", una funzione, una composizione o un trait piccolo possono essere piu semplici.

Riepilogo

Questi strumenti servono a organizzare progetti piu grandi. Se una pagina con classi semplici e metodi chiari risolve il problema, resta su quella strada. La semplicita e una scelta tecnica, non una mancanza.